

SoF Client: Quad & Debris Cvars

Reverse-engineering notes (Linux 1.06a client, IDA). April 2026.

ABSTRACT

This report summarizes how client cvars control effect quads, debris, and FX simulation, and how they relate to CPU cost. Function names and behavior are taken from the sof-bin Linux client (patch 1.06a) in Hex-Rays decompilation.

1. Executive summary

`fx_maxdebrisonscreen` limits how many debris pieces are submitted to the renderer each frame. `fx_sim_time` sets how many FX simulation sub-steps run per frame while catching up effect time.

`cl_max_debris` and `cl_max_quads` cap ring-buffer scans and allocation pools.

`cl_freezequads` (with `freezeworld`) blocks new quad/debris allocation.

Renderer cost for debris is dominated by pushing entities into the refresh list (`V_AddEntity`) and downstream mesh/Ghoul work - not the small memcpys inside `V_AddEntity` itself. Simulation cost scales with `fx_sim_time` because `RunQuadThinkFuncs` runs once per sub-step.

2. Frame pipeline (where things run)

`CL_AddEntities` ends by calling `FX_AddEffects`. Inside `FX_AddEffects`, order matters:

- `FX_AddQuads` runs first: mark/decal quads, debris ring scan, then the `active_quads` list. Debris uses `V_AddEntity` when within the on-screen cap.

- Then `fxFrameTime = 1.0 / fx_sim_time`.

- A loop runs while client time is ahead of `fxCurTime`: `CEffectSystem::update`, `RunQuadThinkFuncs`, `FX_UpdateWeaponIdles`, then `fxCurTime += fxFrameTime` (clamped).

`FX_AddQuads` therefore sees pre-catch-up state for that call; `fxCurTime` advances afterward.

3. Cvar reference

3.1 `fx_maxdebrisonscreen`

Registered in `FX_Init`, default 16.

Used in `FX_AddQuads` only, when iterating the `debrisQuads` ring (size from `cl_max_debris`). For each slot with positive alpha: if fewer than N debris have already been submitted this frame, the client calls `V_AddEntity`; otherwise it clears that quad's alpha (drops it from display).

Cost: At 0, no debris `V_AddEntity` calls - only cheap comparisons and alpha clears. At 1, one debris piece per frame enters the full refresh-entity path (downstream Ghoul/render work).

Higher values multiply that draw-side cost up to the cap each frame.

Does not change how many FX simulation sub-steps run; see `fx_sim_time`.

3.2 `fx_sim_time`

Registered in `FX_Init`, default 60.

Meaning: Treated as an effective FX tick rate (Hz). The client sets `fxFrameTime = 1.0 / fx_sim_time`.

Used in FX_AddEffects: the while loop advances fxCurTime in steps of fxFrameTime until it catches client time (with a snap if FX lags more than ~0.2 s).

Cost: Higher fx_sim_time means smaller step, so more loop iterations per wall-clock frame for the same time debt - more calls per frame to CEffectSystem::update and RunQuadThinkFuncs. Lower values coarsen simulation but reduce CPU.

3.3 cl_max_debris

Registered in FX_Init as cl_max_debris, default 128.

Used in FX_AddQuads and RunQuadThinkFuncs: both iterate the debris ring modulo this count (hard-capped at 1024 in code).

Cost: Wider ring means more slots scanned every frame in FX_AddQuads and every FX sub-step in RunQuadThinkFuncs (for callbacks on live debris). Does not by itself cap on-screen debris; fx_maxdebrisonscreen does that at submit time.

3.4 cl_max_quads

Registered in FX_Init, default 256 (symbol cl_maxQuads in decompilation).

Used in FX_AddQuads and RunQuadThinkFuncs for the markQuads array (decals/mark quads), capped at 1024.

Cost: Higher values increase per-frame work for V_AddQuad on marks and for mark think/update passes. Independent of debris caps.

Note: A separate cvar named cl_quads was not found in FX_Init in this binary; mods may differ.

3.5 cl_freezequads (and freezeworld)

Registered in FX_Init as cl_freezequads.

Used in FX_AllocNewQuad, FX_AllocNewDebrisQuad, and RunQuadThinkFuncs (early-out when frozen). When freeze conditions hold, new general quads and new debris quads fail to allocate; running thinks are skipped.

Cost: Reduces new effect allocation and simulation work when enabled; orthogonal to draw caps.

4. V_AddEntity (renderer handoff)

V_AddEntity copies 96 bytes from the source blob into r_entities, appends a pointer in r_entities_list, increments r_numentities (max 512 entries in the disassembly guard).

Callers include CL_AddPacketEntities (network entities), FX_AddQuads (debris), CL_AddViewWeapon, and particle updates.

The expensive part is not the copy: V_RenderView later passes entity counts and lists into the renderer (function-pointer table re[...]), which drives culling, lighting, and GhoulInst::Render-style paths for mesh entities.

5. Ghoul debris simulation

Debris often carries an IGhoulInst on the quad (e.g. offset 0x188 in HandleDebris / FX_QuadGhoulUpdate paths). FX_QuadGhoulUpdate performs movement, collision (FX_AttemptMove), matrix/color updates via vtable calls - substantial CPU while the quad is alive, mostly independent of whether that frame's V_AddEntity submitted it (visibility cap).

6. How costs combine (matrix)

fx_maxdebrisonscreen - Debris draw: V_AddEntity count + downstream render

fx_sim_time - FX sim: iterations of RunQuadThinkFuncs / CEffectSystem::update per frame

cl_max_debris - Scan + think work on debris ring (width of iteration)

cl_max_quads - Mark/decal V_AddQuad + mark thinks

cl_freezequads - Stops new quads/debris; skips thinks when frozen

Typical tuning: lower fx_maxdebrisonscreen to cut draw entities; lower fx_sim_time to cut simulation frequency; lower cl_max_debris to shrink ring work if many slots are touched.

7. Disclaimer

Behavior is inferred from one Linux client build. Other platforms, patches, or mods may differ. Verify in your own binary if precision matters for competitive or compatibility decisions.